

Contents

Rows instead of renderers	1
Overview	1
Part I — Orientation	2
1. What a Glazed command is	2
2. The command surface today	3
3. What the framework gives you	3
4. The five concepts, in the order you meet them	4
Part II — The design	4
5. Classifying the nineteen verbs	4
6. The client section	6
7. The output layer, and what changes for users	7
8. The exit-code problem	8
9. Streaming: <code>tail --follow</code>	9
10. The row shape is a public API	10
11. Before and after, at the terminal	10
12. What breaks, and how you find out	11
Part III — What to build	12
13. File layout	12
14. API reference	13
15. The six phases	15
16. The tests, and how to verify each by breaking it	16
Part IV — Reference	16
17. Decision records	16
18. Flag compatibility matrix	18
19. Failure modes to watch for	18
20. Open questions	19
21. Where to start reading the code	19

Rows instead of renderers

Overview

`pkg/cli/output.go` is 162 lines that turn API responses into text. It has a `--output` flag with three values, two table renderers built on `tabwriter`, a truncation helper, and a JSON encoder configured three different ways. Every command that returns anything calls into it.

This ticket deletes that file and replaces what it does with Glazed commands. A Glazed command does not render; it **emits rows**, and the output layer — `--output`, `--fields`, `--jq`, `--sort-by`, `--template`, `--output-file` — belongs to the framework. The immediate payoff is that `datadrop` query gains CSV, YAML, Markdown, Excel, field selection, `jq` filtering and templating without a line of rendering code. The longer-term payoff is that the *shape* of what a command returns becomes a declared thing rather than a `Fprintf` format string.

There is a cost, and this guide spends most of Part II on it. Three properties of the current CLI are load-bearing and none of them survives a naive conversion: the documented **exit codes**, the **NDJSON** output format, and the fact that `export` **streams bytes the server formatted**. Each is handled explicitly below.

This guide is for someone joining the project who has to do the conversion. It assumes no Glazed knowledge. It has four parts:

- **Part I** orients you: what a Glazed command is, what the command surface looks like today, and what the framework actually gives you.
 - **Part II** is the design: how each of the nineteen verbs is classified, the three hard problems, and the row shape.
 - **Part III** is what to build: file layout, API reference, six phases, tests.
 - **Part IV** is reference: decision records DR-74 ... DR-84, a complete before/after flag matrix, failure modes and open questions.
-

Part I — Orientation

1. What a Glazed command is

A Glazed command is a struct that describes its own flags and then produces data rather than text. There are three interfaces, and picking the right one per verb is most of the design work (§5).

[// github.com/go-go-golems/glazed/pkg/cmds](https://github.com/go-go-golems/glazed/pkg/cmds)

```
type GlazeCommand interface {
    RunIntoGlazeProcessor(ctx context.Context, vals *values.Values, gp middlewares.Processor) error
}

type WriterCommand interface {
    RunIntoWriter(ctx context.Context, vals *values.Values, w io.Writer) error
}

type BareCommand interface {
    Run(ctx context.Context, vals *values.Values) error
}
```

GlazeCommand is the interesting one. `gp.AddRow(ctx, row)` accepts a `types.Row` — an ordered map — and everything downstream of that call is the framework’s problem: which columns to show, in what order, in what format, to which file, filtered by what expression.

The command declares its flags as **fields** grouped into **sections**, and reads them back through a typed struct:

```
type QuerySettings struct {
    Drop string `glazed:"drop"`
    Limit int    `glazed:"limit"`
    Order string `glazed:"order"`
}

func (c *QueryCommand) RunIntoGlazeProcessor(
    ctx context.Context, vals *values.Values, gp middlewares.Processor,
) error {
    s := &QuerySettings{}
    if err := vals.DecodeSectionInto(schema.DefaultSlug, s); err != nil {
        return err
    }
    ...
    for _, e := range result.Events {
        if err := gp.AddRow(ctx, rowForEnvelope(e)); err != nil {
            return err
        }
    }
}
```

```

    return nil
}

```

Note what is absent: no `--output` handling, no format switch, no `tabwriter`, no `json.Encoder`. That is the whole proposition.

2. The command surface today

Nineteen leaf verbs across five files in `pkg/cli/`, totalling 2 486 lines.

Verb	File	Produces	Today's output
serve	serve.go	a running server	log lines
create DROP	push.go	a drop	renderJSON
list	push.go	a set of drops	renderDrops — table/json/ndjson
inspect DROP	push.go	one drop's metadata	renderJSON
push DROP k=v...	push.go	one accepted event	renderJSON
query DROP	read.go	a set of events	renderEvents — table/json/ndjson
tail DROP [--follow]	read.go	a set of events, streaming	renderEvent per event
export DROP	read.go	bytes the server formatted	io.Copy
schema put DROP	read.go	one schema version	renderJSON
schema show DROP	read.go	one schema	renderJSON
dataset push	dataset.go	one dataset version	renderJSON
dataset list DROP	dataset.go	a set of datasets	renderJSON
dataset show	dataset.go	one dataset or version	renderJSON
dataset get	dataset.go	files on disk	progress to stderr
dataset rm	dataset.go	a deletion count	renderJSON
dataset import	dataset.go	an ingest count	renderJSON
dataset gc	dataset.go	a reclaim count	renderJSON
whoami	whoami.go	one principal	renderJSON
healthcheck	healthcheck.go	an exit code	nothing

Read the “Produces” column rather than the “Verb” column. Five verbs produce a **set of records** and are the reason to do this ticket at all. Nine produce **one record**, which is a set of size one. Three produce something that is not records. Two produce nothing.

Note also that eleven verbs call `renderJSON`, whose entire body is “encode this with two-space indent”. Those eleven have no table output today at all: `datadrop dataset list mydrop` prints JSON whether you asked for it or not, and `--output table` is silently ignored. That is not a design; it is eleven places where nobody wanted to write another `tabwriter` block.

3. What the framework gives you

Run any of these against a converted command:

```

$ datadrop query greenhouse --output csv
$ datadrop query greenhouse --output yaml
$ datadrop query greenhouse --output markdown
$ datadrop query greenhouse --fields seq,time,data.temp_c
$ datadrop query greenhouse --jq 'select(.data.temp_c > 21)'
$ datadrop query greenhouse --sort-by -seq
$ datadrop query greenhouse --select data.temp_c

```

```
$ datadrop query greenhouse --template '{{.seq}} {{.data.temp_c}}'
$ datadrop query greenhouse --output excel --output-file readings.xlsx
```

None of those require code in datadrop. `--output` accepts table, csv, tsv, json, yaml, sql, template, markdown and excel; `--table-format` picks ascii, markdown, html, csv or tsv within the table renderer; `--stream` switches to row-at-a-time emission.

A warning about two flag names that read as each other's opposite, because you will reach for the wrong one: **--jq filters rows** and **--filter removes columns**. `--filter b` drops the `b` column; `--jq 'select(.b > 22)'` drops the rows where the predicate is false. `--select b` prints one field per line with no header, which is the `cut -f` of this toolchain. All three were verified against glaze v1.3.8 while writing this guide, after the first draft of it used `--filter` as a row predicate and was wrong.

Beyond those, and also free: `--rename`, `--rename-regexp`, `--reorder-columns`, `--sort-columns`, `--remove-nulls`, `--remove-duplicates`, `--flatten`, `--glazed-limit`, `--glazed-skip`, `--regex-fields` and `--field-jq`.

Two things come free with the section machinery as well: `--print-schema` dumps the command's field definitions, and `--print-parsed-fields` shows what each flag resolved to and from which source. On a CLI where `--addr` is reachable from a flag, an environment variable and a config file, that second one ends a whole category of support question.

4. The five concepts, in the order you meet them

Field. One flag or one positional argument. `fields.New("limit", fields.TypeInteger, fields.WithDefault(100), fields.WithHelp("..."))`. A positional argument is a field with `fields.WithIsArgument(true)`, declared through `cmds.WithArguments` rather than `cmds.WithFlags`.

Section. A named group of fields that can be attached to many commands. `settings.NewGlazedSchema()` is the output section; `logging.NewLoggingSection()` is the one this repository already uses at the root. §6 adds a datadrop client section for `--addr` and `--token`.

Values. What the parser produces. `vals.DecodeSectionInto(slug, &settings)` fills a struct through `glazed:"..."` tags. Never read cobra flags directly inside a Glazed command — the value may have come from a config file or a profile, and the cobra flag would be empty.

Row. `types.NewRow(types.MRP("seq", e.Seq), types.MRP("time", e.Time))`. An ordered map. MRP is “make row pair”. It returns a value, not a pointer, and `AddRow` takes a value: a helper declared as returning `*types.Row` will not compile against it.

Processor. `gp.AddRow(ctx, row)`. Everything after that is the framework.

Part II — The design

5. Classifying the nineteen verbs

DR-74. Classify each verb by what it produces, not by what it is.

The question to ask is not “is this an important command” but “what does a caller do with the output”. Three answers, three interfaces:

- If the output is **a set of records** a caller would want to filter, sort, select columns from, or load into a spreadsheet — `GlazeCommand`.
- If the output is **a byte stream whose format is already decided** — a file download, a server-rendered CSV — `WriterCommand`. Turning it into rows means parsing something in order to re-serialise it.
- If the output is **a side effect** and the exit code is the result — `BareCommand`.

Applying that:

Interface	Verbs	Count
GlazeCommand	list, query, tail, inspect, whoami, create, push, schema put, schema show, dataset list, dataset show, dataset push, dataset rm, dataset import, dataset gc	15
WriterCommand	export	1
BareCommand	serve, healthcheck, dataset get	3

5.1 A single object is a one-row set

Nine of the fifteen return one thing: `whoami` returns a principal, `inspect` returns a drop, `push` returns an accepted event. Emitting one row rather than `renderJSON` looks like ceremony and is not, for three reasons.

It makes `--output json` mean the same thing everywhere. Today `whoami` prints an indented object and `list` prints an array, and a script has to know which.

It makes those commands composable. `datadrop whoami --fields user_id` is a useful thing to be able to type, and today it requires `jq`.

It costs one function. The one-row commands share a shape:

```
func (c *WhoamiCommand) RunIntoGlazeProcessor(ctx context.Context, vals *values.Values, gp middleware.GlazeProcessor, me, err := api.Whoami(ctx)
    if err != nil {
        return exitOn(err) // $8
    }
    return gp.AddRow(ctx, types.NewRow(
        types.MRP("principal", me.Principal),
        types.MRP("kind", me.Kind),
        types.MRP("scopes", me.Scopes),
    ))
}
```

5.2 export stays bytes

DR-75. export remains a WriterCommand. The server formats it.

This is the classification that is easiest to get wrong, because `export` produces exactly the kind of tabular data Glazed exists for. Look at what the command actually does (`pkg/cli/read.go`, `newExportCmd`):

```
body, err := api.Export(ctx, drop, format, q) // GET /v1/drops/X/export?format=csv
defer body.Close()
_, err = io.Copy(sink, body)
```

The CLI never sees a record. It opens a response body and copies it. The formatting happens on the server, in `pkg/tabular`, which is also what the web UI and any `curl` user get.

Converting it to a `GlazeCommand` would mean fetching NDJSON, parsing it into rows, and re-serialising it as CSV on the client. That:

- moves the export format definition from one place (the server) to two, so `curl .../export?format=csv` and `datadrop export --output csv` can disagree;

- loses streaming. `io.Copy` is constant-memory over a 400 MB export; a Glazed table formatter buffers every row to compute column widths unless `--stream` is set, and even streaming pays a parse-and-re-encode per row;
- gains nothing a caller wants. Someone running `datadrop export` is asking for the server's canonical export, not for a client-side view of it.

`export` keeps its own `--format csv|ndjson|json` flag, which names a *server-side* format and is a different thing from `--output`. §7.3 says how to keep that from being confusing.

5.3 The three bare commands

DR-81. `serve`, `healthcheck` and `dataset get` stay bare.

`serve` runs until it is stopped and its output is log lines. `healthcheck` exists to produce an exit code for a container runtime. `dataset get` writes files to disk and reports progress on `stderr`; its result is on the filesystem.

The temptation with `dataset get` is to emit one row per downloaded file. That is worth doing **as well**, not instead — see the open question in §20.

6. The client section

DR-76. `--addr` and `--token` become a Glazed section, not persistent cobra flags bound to a struct.

Today `globalOptions` is a struct populated by `PersistentFlags().StringVar`, threaded into every constructor as `newXCmd(opts)`. That works and it is the reason every command signature carries a parameter nothing else needs.

As a section:

```
// pkg/cli/section.go
const ClientSectionSlug = "datadrop-client"

type ClientSettings struct {
    Addr string `glazed:"addr"`
    Token string `glazed:"token"`
}

func NewClientSection() (schema.Section, error) {
    return schema.NewSection(
        ClientSectionSlug,
        "How to reach the datadrop server",
        schema.WithFields(
            fields.New("addr", fields.TypeString,
                fields.WithDefault("http://localhost:8080"),
                fields.WithHelp("datadrop server base URL [DATADROP_ADDR]")),
            fields.New("token", fields.TypeString,
                fields.WithHelp("bearer token [DATADROP_TOKEN]")),
        ),
    ),
}

func clientFrom(vals *values.Values) (*client.Client, error) {
    s := &ClientSettings{}
    if err := vals.DecodeSectionInto(ClientSectionSlug, s); err != nil {
        return nil, err
    }
}
```

```
    return client.New(s.Addr, s.Token)
}
```

Three things this buys beyond tidiness:

- **Config files and profiles.** A section can be filled from a YAML file or a named profile, so `datadrop --profile staging query greenhouse` becomes possible without `datadrop` implementing precedence itself.
- **--print-parsed-fields explains itself.** “Where did this `--addr` come from” becomes a flag rather than a bug report.
- **The section is attached per command, and only commands that talk to a server get it.** `serve` does not need `--token`; today it has one, because the flag is persistent on the root.

The environment fallback needs the same care the logging section needed (`pkg/cli/root.go, addLoggingSection`): Glazed sections do not read the environment by default. Either supply `AppName` on the parser config, which enables the built-in `DATADROP_*` env source, or set the values explicitly. **If you pass a custom `MiddlewaresFunc` you replace the default chain and lose env loading**, which is the specific way this gets broken silently.

7. The output layer, and what changes for users

This is where the ticket is user-visible, and it needs to be deliberate rather than incidental.

7.1 --output ndjson has no exact equivalent

DR-78. Retire `--output ndjson` loudly, with an alias and a message. Do not map it silently onto something that is nearly the same.

Today: `--output ndjson` prints one compact JSON object per line.

Glazed’s `--output` choices are `table`, `csv`, `tsv`, `json`, `yaml`, `sql`, `template`, `markdown`, `excel`. There is no `ndjson`. The closest is:

```
$ datadrop query greenhouse --output json --output-as-objects
{
  "seq": 1,
  "time": "...
}
{
  "seq": 2,
  "time": "...
}
```

That is a stream of concatenated JSON values. `jq` reads it correctly, and so does any streaming JSON parser. But it is **not line-oriented**: a script doing

```
datadrop query greenhouse --output ndjson | while read -r line; do
  echo "$line" | jq -r .seq
done
```

silently produces garbage, because a “line” is now `{` or `"seq": 1,.`

So: keep `ndjson` as an accepted value for one release, have it print a warning to `stderr` naming the replacement, and map it to `--output json --output-as-objects`. Anyone piping into `jq` is unaffected; anyone reading lines is told, in the one place they will see it.

For the strictly line-oriented case, `--output template --template '{{ toJson . }}'` produces exactly one compact object per line. Document that in the message.

7.2 --output becomes per-command, not global

Today --output is a persistent flag on the root and defaults to table for everything, including the eleven commands that ignore it. After the conversion it comes from the Glazed output section, attached to the fifteen Glaze commands. `serve --output json` stops parsing, which is correct — it never did anything.

7.3 export --format and --output are different things

They will be confused, so the help has to say which is which:

- --format names the **server-side** export format. The bytes are produced by pkg/tabular on the server and streamed through.
- --output names the **client-side** rendering of rows the command emitted.

export has --format and no --output. Every Glaze command has --output and no --format. No command has both, and the guide's rule for a reviewer is that a command with both flags is a command that has been classified wrong.

7.4 The full compatibility matrix

Part IV §18 lists every flag before and after. The short version: --addr, --token, --limit, --from, --to, --after, --order, --follow and --format all keep their names and meanings. --output keeps its name and gains six values while losing one. Nothing else moves.

8. The exit-code problem

DR-77. The documented exit codes survive by exiting before returning.

This is the hardest problem in the ticket and it is entirely invisible until you run the smoke tests.

pkg/cli/root.go documents five exit codes as part of the CLI contract, and cmd/datadrop/smoke_test.go:226 asserts them:

```
const (
    ExitOK = 0; ExitError = 1; ExitUsage = 2
    ExitAuth = 3; ExitNotFound = 4; ExitValidation = 5
)
```

Execute() maps an *client.APIError onto them by status code, so a script can branch on why a command failed without parsing stderr.

Now read what Glazed's cobra builder does (glazed/pkg/cli/cobra.go, runCobraCommand):

```
cmd.Run = func(cmd *cobra.Command, args []string) {
    ...
    err = runFunc(ctx, parsedValues)
    if _, ok := err.(*cmds.ExitWithoutGlazeError); ok {
        os.Exit(0)
    }
    cobra.CheckErr(err)    // ← prints "Error: ..." and os.Exit(1)
}
```

It sets cmd.Run, not cmd.RunE. The error never returns to Execute(). cobra.CheckErr exits 1 for everything. There is no option to change this: commandBuildConfig has five fields and none of them is an error hook.

So a naive conversion turns 3, 4 and 5 into 1, changes the message prefix from datadrop: to Error:, and fails TestExitCodes — which is the good outcome, because the alternative is shipping it.

The fix inside this ticket is a helper every command's error path goes through:


```
// pkg/cli/exit.go

// exit0n maps an error onto the documented exit codes and exits.
//
// Glazed's cobra builder ends every command with cobra.CheckErr, which exits 1
// unconditionally and never returns the error to Execute(). Our exit codes are
// part of the CLI contract (scripts branch on them), so a command that wants to
// preserve them has to exit before returning. Errors that map to ExitError are
// handed back instead, so cobra keeps doing the ordinary thing with them.
func exit0n(err error) error {
    if err == nil {
        return nil
    }
    code := exitCodeFor(err)
    if code == ExitError {
        return err
    }
    _, _ = fmt.Fprintln(os.Stderr, "datadrop: "+err.Error())
    os.Exit(code)
    return nil // unreachable
}
```

Every `return err` in a converted command that could carry an `*APIError` becomes `return exit0n(err)`. It is mechanical, it is ugly, and it is honest about why it exists.

The better fix, and it is not in this ticket, is a `WithExitCodeFunc` option on Glazed's builder so an application can supply the mapping once. §20 records it as the preferred long-term answer; doing it means a glazed PR, which is a different ticket in a different repository.

A test pins the behaviour either way — see §16.

9. Streaming: `tail --follow`

DR-80. `tail --follow` emits one row per event and defaults `--stream` to `true`.

`tail --follow` subscribes to the drop's SSE stream and prints each event as it arrives. A Glaze command can do that: `gp.AddRow` may be called any number of times over any duration.

What it must not do is buffer. The default table formatter collects every row before printing, because it computes column widths from the whole set — which for a `--follow` that never ends means printing nothing, forever. That is a particularly bad failure because it looks exactly like “no events are arriving”.

So the command sets a per-command output default:

```
glazedSection, err := settings.NewGlazedSchema(
    settings.WithOutputSectionOptions(
        schema.WithDefaults(map[string]interface{}{"stream": true}),
    ),
)
```

The reader can still pass `--stream=false` and get the buffering behaviour, and for a bounded `tail` without `--follow` that is a reasonable thing to want. Setting the *default* is what stops the common invocation from hanging.

Two more things `--follow` needs and gets for free from the builder: the context is already wired to `SIGINT/SIGTERM` by `runCobraCommand`, so `Ctrl-C` ends the loop cleanly, and returning `cmds.ExitWithoutGlazeError` exits 0 rather than reporting an error, which is the right shape for “the user stopped it”.

10. The row shape is a public API

DR-79. Name the fields once, in one function per response type, and pin them with a test.

The moment `datadrop query --fields seq,time,data.temp_c` works, those names are something scripts depend on. Renaming `seq` to `sequence` is then a breaking change, and it is a breaking change that no compiler catches.

So each response type gets exactly one projection function, and they live together:

```
// pkg/cli/rows.go
func rowForEnvelope(e datadrop.Envelope) types.Row
func rowForDrop(d datadrop.Drop) types.Row
func rowForDataset(d datadrop.Dataset) types.Row
func rowForPrincipal(p client.Me) types.Row
```

and `pkg/cli/rows_test.go` asserts the exact key set of each. That test is a change detector, and it earns its keep because the thing it detects is a contract rather than a detail.

10.1 Payload columns are flattened with a data. prefix

DR-83. An event’s payload is flattened into data.* columns, using the server’s existing projection rather than a second one.

`pkg/tabular/table.go:168-178` already does this:

events, in order. Payload columns follow, prefixed “data.” and sorted.

```
const DataPrefix = "data."
```

That projection is what the server’s `/table` endpoint returns and therefore what the **web workbench** shows: a field chip in the workbench reads `data.temp_c`. If the CLI invented its own flattening — or emitted data as a nested blob — then `datadrop query --fields data.temp_c` and the column the workbench names would be two different things that happen to look alike.

So: call `pkg/tabular`. Do not write a second flattener. This is the single strongest consistency argument in the ticket, and it also means the CLI’s column names are already documented by the web UI’s help pages.

A nested data object remains available through `--output json` without `--fields`, because the JSON formatter emits whatever the row holds; the flattening happens in the projection, so what you get is flat there too. If a caller genuinely wants the original nested envelope, that is what `export --format ndjson` is for, and saying so in the help is cheaper than a flag.

11. Before and after, at the terminal

The point of the ticket, in the form a reader will actually meet it.

datadrop list

BEFORE

```
$ datadrop list
NAME          CREATED              RET
greenhouse    2026-07-20T09:14:02Z 30d
sensors       2026-07-22T11:40:55Z -
```

AFTER (identical by default)

\$ datadrop list		
name	created	retention
greenhouse	2026-07-20T09:14:02Z	30d
sensors	2026-07-22T11:40:55Z	

datadrop dataset list — eleven commands gain a table they never had

BEFORE

```
$ datadrop dataset list sensors
[
  {
    "name": "readings",
    "version": 3,
    "files": 4,
    "published_at": "2026-07-24T18:02"
  },
  ...
]
```

AFTER

```
$ datadrop dataset list sensors
```

name	version	files	published
readings	3	4	2026-07-24T18:02
census	1	1	2026-07-19T08:40

```
$ datadrop dataset list sensors --output json
[ ... the same JSON, when you ask for it ... ]
```

--output table was accepted and ignored.

What query can suddenly do

```
$ datadrop query greenhouse --limit 3 --fields seq,time,data.temp_c
```

seq	time	data.temp_c
184	2026-07-26T18:04:11.512Z	21.7
185	2026-07-26T18:04:41.008Z	21.9
186	2026-07-26T18:05:10.774Z	22.1

```
$ datadrop query greenhouse --output csv --fields seq,data.temp_c
seq,data.temp_c
184,21.7
185,21.9
186,22.1
```

```
$ datadrop query greenhouse --template '{{.seq}} {{.data.temp_c}}C'
184 21.7C
185 21.9C
186 22.1C
```

```
$ datadrop query greenhouse --output excel --output-file july.xlsx
```

```
$ datadrop tail greenhouse --follow --fields time,data.temp_c
2026-07-26T18:06:02.114Z 22.0
2026-07-26T18:06:31.902Z 22.2
^C
```

Every one of those is zero lines of datadrop code.

12. What breaks, and how you find out

Four test files assert on CLI behaviour, and three of them will fail. That is the design working: each failure is a decision that needs making rather than an inconvenience.

Test	What it asserts	What happens
smoke_test.go:190	tail output contains SEQ	Fails. The table header is now lower-case seq. Decide: change the assertion, or set column titles.
smoke_test.go:226 TestExitCodes	3 / 4 / 5 for auth / not-found / validation	Fails without \$8's exit0n. This is the test that catches the whole problem.
smoke_test.go:311 TestDiagnosticsStayOffStdout	stdout stays clean at --log-level debug	Must keep passing. Glazed writes rows to stdout and logs to stderr, so it should — verify rather than assume.
dataset_smoke_test.go	parses renderJSON output	Fails where output becomes a table. Add --output json to those invocations.
pkg/cli/push_test.go	key=value parsing	Unaffected — it tests the parser, not the output.

On the header case: prefer lower-case field names and change the assertion. seq is what --fields seq takes and what --output json emits; a table that shouts SEQ while every other surface says seq is a third spelling of one thing.

Part III — What to build

13. File layout

DR-82. The directory structure mirrors the command tree.

If you type `datadrop dataset list`, then `dataset` is a directory and `list` is a file in it.

```

pkg/cli/
  root.go          the root command, logging section, help system (exists)
  section.go       NewClientSection, ClientFrom (new)
  exit.go          exit0n, exitCodeFor (moved out of root.go)
  rows.go          rowForEnvelope, rowForDrop, rowForDataset, ... (new)
  rows_test.go     the key-set contract (new)
  serve.go         BareCommand (rewritten)
  healthcheck.go   BareCommand (rewritten)
  drops/
    root.go        registers create, list, inspect, push
    create.go list.go inspect.go push.go
  events/
    root.go        registers query, tail, export
    query.go tail.go export.go
  schemacmd/
    root.go        registers put, show
    put.go show.go
  dataset/
    root.go        registers push, list, show, get, rm, import, gc
    push.go list.go show.go get.go rm.go import.go gc.go

```

`pkg/cli/output.go` is deleted. `read.go`, `push.go` and `dataset.go` are dissolved into the directories above.

Three notes. create, list, inspect and push stay **top-level verbs** — datadrop list, not datadrop drops list — because that is the existing surface and this is not a renaming ticket; the drops/directory groups them in the source without grouping them in the CLI. Each root.go exposes Register(root *cobra.Command) error and adds its commands wherever they belong. And the group root.go files are the only places cli.BuildCobraCommandFromCommand is called, so no verb file contains cobra wiring. The package is schemacmd, not schema, because glazed/pkg/cmds/schema is imported by every verb file in it.

14. API reference

14.1 The shape every verb file follows

package events

```
type QueryCommand struct{ *cmds.CommandDescription }

type QuerySettings struct {
    Drop string `glazed:"drop"`
    Limit int `glazed:"limit"`
    From string `glazed:"from"`
    To string `glazed:"to"`
    After int64 `glazed:"after"`
    Order string `glazed:"order"`
}

func NewQueryCommand() (*QueryCommand, error) {
    glazedSection, err := settings.NewGlazedSchema()
    if err != nil { return nil, err }

    clientSection, err := ddcli.NewClientSection()
    if err != nil { return nil, err }

    cmdSettings, err := glazedcli.NewCommandSettingsSection()
    if err != nil { return nil, err }

    return &QueryCommand{cmds.NewCommandDescription(
        "query",
        cmds.WithShort("Query events from a drop"),
        cmds.WithLong(`... with runnable examples ...`),
        cmds.WithArguments(
            fields.New("drop", fields.TypeString,
                fields.WithIsArgument(true),
                fields.WithHelp("The drop to read")),
        ),
        cmds.WithFlags(
            fields.New("limit", fields.TypeInteger,
                fields.WithDefault(datadrop.DefaultLimit),
                fields.WithHelp("Maximum events to return")),
            fields.New("order", fields.TypeChoice,
                fields.WithChoices("asc", "desc"),
                fields.WithDefault("desc"),
                fields.WithHelp("Sequence order")),
        ),
        cmds.WithSections(glazedSection, clientSection, cmdSettings),
    )}, nil
}
```

```

func (c *QueryCommand) RunIntoGlazeProcessor(
    ctx context.Context, vals *values.Values, gp middlewares.Processor,
) error {
    s := &QuerySettings{}
    if err := vals.DecodeSectionInto(schema.DefaultSlug, s); err != nil {
        return err
    }
    api, err := ddcli.ClientFrom(vals)
    if err != nil { return err }

    q, err := queryFrom(s)
    if err != nil { return err } // usage error

    result, err := api.Query(ctx, q)
    if err != nil { return exitOn(err) } // API error: mapped exit code

    for _, e := range result.Events {
        if err := gp.AddRow(ctx, ddcli.RowForEnvelope(e)); err != nil {
            return err
        }
    }
    return nil
}

```

14.2 Registration

```

// pkg/cli/events/root.go
func Register(root *cobra.Command) error {
    builders := []func() (cmds.Command, error){
        func() (cmds.Command, error) { return NewQueryCommand() },
        func() (cmds.Command, error) { return NewTailCommand() },
        func() (cmds.Command, error) { return NewExportCommand() },
    }
    for _, build := range builders {
        c, err := build()
        if err != nil { return err }
        cobraCmd, err := glazedcli.BuildCobraCommandFromCommand(c,
            glazedcli.WithParserConfig(glazedcli.CobraParserConfig{
                ShortHelpSections: []string{schema.DefaultSlug},
                AppName:            "datadrop", // enables DATADROP_* env loading
            }),
        )
        if err != nil { return err }
        root.AddCommand(cobraCmd)
    }
    return nil
}

```

ShortHelpSections: []string{schema.DefaultSlug} is what keeps datadrop query --help showing the command's own flags rather than fifty output flags. The full set stays reachable through --long-help.

14.3 Positional arguments

datadrop push DROP key=value key=value ... has one required positional and a variadic tail. Two rules from the field system:

- A variadic positional needs `fields.TypeStringList` with `fields.WithIsArgument(true)`, and it must be **last**. Only one list argument is allowed.
- A `fields.TypeString` positional takes exactly one value.

```
cmds.WithArguments(
    fields.New("drop", fields.TypeString, fields.WithIsArgument(true),
        fields.WithHelp("The drop to append to")),
    fields.New("pairs", fields.TypeStringList, fields.WithIsArgument(true),
        fields.WithHelp("key=value pairs forming the payload")),
)
```

15. The six phases

Each phase ends green: `GOWORK=off make lint, GOWORK=off go test ./...`, and a manual run of the verbs it touched.

Phase 1 — The scaffolding, nothing user-visible

`section.go` (the client section), `exit.go` (`exit0n`), `rows.go` with the four projection functions, `rows_test.go` pinning their key sets. Nothing is wired to a command yet.

Why first: every later phase consumes all four, and the row shapes are the part worth arguing about before fifteen commands depend on them.

Acceptance: `rows_test.go` passes, and fails when a key is renamed.

Phase 2 — One verb, end to end: list

Convert `datadrop list` alone. Keep every other command as it is; the two styles coexist on one root without trouble.

Why second: `list` is the smallest command that returns a set, it already has a table today so the before/after is comparable, and it forces every wiring question — section attachment, parser config, help rendering, `--print-schema` — to be answered once, on a command small enough to throw away.

Acceptance: `datadrop list` renders a table; `--output json, --output csv, --fields name` and `--sort-by name` all work; `datadrop list --help` shows the command's flags and not the whole output section.

Phase 3 — The reading verbs: query, tail, export, inspect, whoami

The set-returning commands, plus `export` as a `WriterCommand` — deliberately in one phase, because doing them together is what makes the `--format` versus `--output` distinction concrete rather than theoretical.

`tail --follow` gets the streaming default (`$9`) and `ExitWithoutGlazeError` on interrupt.

Acceptance: the `$11` transcript, verbatim. `datadrop tail X --follow` prints rows as they arrive rather than buffering. `Ctrl-C` exits 0.

Phase 4 — The exit-code contract

`exit0n` threaded through every converted command, and `TestExitCodes` green again.

Why its own phase: it is a behaviour that is easy to half-do, and a phase boundary is where you check it rather than where you notice it.

Acceptance: a 401 exits 3, a 404 exits 4, a 422 exits 5, everything else exits 1, and the message keeps its `datadrop:` prefix.

Phase 5 — Writes and datasets: the remaining ten verbs

create, push, schema put, schema show, and dataset × 7. Mechanical once phases 1–4 are done. dataset get becomes a BareCommand; serve and healthcheck become BareCommands with their flags moved into fields.

Acceptance: dataset_smoke_test.go green, with --output json added where it parses output.

Phase 6 — Delete, document, deprecate

Delete pkg/cli/output.go. Add the ndjson deprecation shim and its warning. Write a help page — datadrop help cli-output — covering the output layer, the --format/--output distinction and the column names, in the same pkg/doc/topics/ tree the web UI pages already live in. Update README.md and deploy/compose/docker-compose.yml if any invocation changed.

Acceptance: grep -r renderEvents pkg/ returns nothing. datadrop help cli-output resolves.

16. The tests, and how to verify each by breaking it

Test	Guards	Break it by
pkg/cli/rows_test.go	the row key sets (DR-79)	Renaming seq to sequence in rowForEnvelope
TestExitCodes	the exit-code contract (DR-77)	Replacing exit0n(err) with err in one verb
a new TestNdjsonDeprecation	the alias still works and still warns	Removing the warning, or the mapping
TestDiagnosticsStayOffStdout	rows on stdout, logs on stderr	Emitting a progress line to stdout
a new TestEveryGlazeCommand-HasClientSection	that no verb forgot --addr	Dropping the section from one constructor

The last one is worth writing even though it sounds like paranoia. A verb that forgets the client section compiles, runs, and fails at the first request with a confusing default-address error — and it is exactly the mistake a copy-paste between verb files produces.

Part IV — Reference

17. Decision records

DR	Decision
DR-74	Classify each verb by what it <i>produces</i> , not by what it is. A set of records is a GlazeCommand; a byte stream whose format is already decided is a WriterCommand; a side effect is a BareCommand. Fifteen, one and three.
DR-75	export stays a WriterCommand. The server formats it, in pkg/tabular, and that is the same code path curl and the web UI get. Converting it to rows would put the export format in two places, lose constant-memory streaming over a 400 MB body, and gain nothing a caller asked for.

DR	Decision
DR-76	--addr and --token become a Glazed section attached per command, not persistent cobra flags bound to a globalOptions struct. Config files, profiles and --print-parsed-fields follow from it, and serve stops carrying a --token flag it never used.
DR-77	The documented exit codes survive by exiting <i>before</i> returning. Glazed's builder sets cmd.Run and ends with cobra.CheckErr, which exits 1 unconditionally and never returns the error to Execute(); commandBuildConfig has no error hook. A local exitOn helper maps and exits; anything that maps to ExitError is handed back so cobra does the ordinary thing.
DR-78	--output ndjson is retired loudly. It has no exact equivalent — --output json --output-as-objects is a stream of concatenated JSON values, which jq reads and while read line does not. Keep the value for one release, warn to stderr, and name both replacements.
DR-79	The row shape is a public API. One projection function per response type in pkg/cli/rows.go, and a test pinning the exact key set of each, because --fields seq makes those names something scripts depend on and no compiler catches a rename.
DR-80	tail --follow defaults --stream to true. The table formatter buffers every row to compute column widths, so a follow that never ends prints nothing, forever — a failure indistinguishable from “no events are arriving”.
DR-81	serve, healthcheck and dataset get stay bare. Their results are a running process, an exit code, and files on disk.
DR-82	The directory structure mirrors the command tree: one directory per group, one file per verb, one root.go per group holding the only cobra wiring. drops/ groups four verbs in the source that stay top-level in the CLI.
DR-83	Event payloads are flattened into data.* columns by calling pkg/tabular, not by writing a second flattener. That projection is what the server's /table endpoint returns and therefore what the web workbench names, so datadrop query --fields data.temp_c and the workbench's field chip are the same column rather than two that look alike.
DR-84	Convert leaf-first, one phase per group, with both styles coexisting on one root between phases. A big-bang conversion of nineteen verbs has no green point in the middle of it.

18. Flag compatibility matrix

Unchanged unless the last column says otherwise.

Flag	Before	After	Note
--addr	persistent, \$DATADROP_ADDR	client section, \$DATADROP_ADDR	Only on commands that talk to a server
--token	persistent, \$DATADROP_TOKEN	client section, \$DATADROP_TOKEN	Same
--log-level and friends	logging section	unchanged	Already converted; see the previous commit
--output table	root persistent	glazed output section	Now actually implemented on eleven more commands
--output json	root persistent	glazed output section	Same shape
--output ndjson	root persistent	removed after one release	→ --output json --output-as-objects, or --output template --template '{{ toJson . }}' for strict NDJSON
--output csv/tsv/yaml/markdown/excel/sql/template	—	new	
--fields, --sort-by, --template, --output-file, --stream, --table-format, --table-style	—	new	
--jq, --select, --filter	—	new	--jq filters rows; --filter removes columns; --select prints one field per line
--rename, --reorder-columns, --remove-nulls, --flatten, --glazed-limit, --glazed-skip	—	new	
--print-schema, -- print-parsed-fields	—	new	From the command settings section
--limit, --from, --to, --after, --order	query, tail, export	unchanged	Now declared as fields
--follow	tail	unchanged	Sets --stream by default
--format	export	unchanged	Server-side format; distinct from --output
-o / --output-file	export	unchanged	Same meaning as glazed's

19. Failure modes to watch for

A --follow that prints nothing. The most likely single defect in this ticket. The table formatter buffers; a stream that never ends never flushes. DR-80 sets the default, but a reader who passes

--output table --stream=false alongside --follow gets the hang, so the help for --follow should say so.

Exit code 1 for everything. Silent unless TestExitCodes runs. Any verb whose error path returns err instead of exit0n(err) loses the mapping for that verb alone, which is worse than losing it everywhere because it looks like it works.

Two flatteners. Someone writes row["data"] = e.Data in one verb and calls tabular in another. Then --fields data.temp_c works on query and returns empty on tail. Route every envelope through RowForEnvelope.

A verb missing the client section. Compiles, runs, and fails at the first request against http://localhost:8080 regardless of --addr, because the flag does not exist so the default stands.

--print-parsed-fields leaking the token. The command settings section prints every resolved field and its source. --token is a resolved field. Check whether Glazed redacts it; if it does not, mark the field or drop the section from commands that carry a credential. This is the one item in the ticket with a security consequence and it should be checked in phase 2, on the first verb, not discovered in phase 5.

Losing DATADROP_ADDR / DATADROP_TOKEN. The env source comes from AppName on the default parser path. Passing a custom MiddlewaresFunc replaces the chain and takes env loading with it. The same trap the logging section had with DATADROP_LOG_LEVEL, one layer over.

A message prefix change nobody decided. cobra.CheckErr prints Error: something. The current CLI prints datadrop: something. Both appear after this conversion depending on which path the error took. Pick one, in phase 4, and make exit0n and Execute() agree.

20. Open questions

1. **Should WithExitCodeFunc be added to Glazed instead?** DR-77's exit0n is a workaround that every go-go-golems CLI with an exit-code contract will need independently. An option on commandBuildConfig that replaces the cobra.CheckErr call would be perhaps thirty lines upstream. That is the right fix; it is a different repository and a different ticket. Doing it would let phase 4 delete exit0n rather than thread it.
2. **Should dataset get also emit a row per downloaded file?** It stays a BareCommand under DR-81 because its result is on the filesystem, but "which files did I just get, and how big were they" is a reasonable question and a GlazeCommand answers it well. The blocker is that its progress output goes to stderr today and rows would go to stdout, which is a behaviour change for anyone redirecting.
3. **Do create, push, schema put, dataset rm/import/gc want a row at all?** They currently print an object. A row is more useful in a pipeline and noisier at a prompt. The alternative is BareCommand plus a log line. This guide assumes rows, on the grounds that a write command that tells you what it wrote is composable and one that prints a sentence is not — but it is a product call.
4. **Should the top-level verbs move under groups?** datadrop drops list rather than datadrop list would match the directory layout exactly and match datadrop dataset list. It is also a breaking change to every script and every line of the README. Out of scope here; worth deciding before the surface grows further.
5. **Does the row shape want a version?** DR-79 makes the column names a contract. Contracts eventually change. Nothing in this ticket provides a way to change them compatibly, and the honest answer may be that a rename is a major-version event.

21. Where to start reading the code

1. pkg/cli/output.go — 162 lines, the whole thing this ticket removes. Read it first so the size of what Glazed replaces is concrete.

2. `pkg/cli/read.go`, `newQueryCmd` and `newExportCmd` — the two commands that define the interesting boundary. One returns records; the other copies bytes.
3. `pkg/cli/root.go` — `exitCodeFor` and `Execute`, which are what §8 is about, and `addLoggingSection`, which is the pattern §6 follows.
4. `pkg/tabular/table.go:160-180` — `DataPrefix` and the payload-column projection that DR-83 says to reuse.
5. `glazed/pkg/cli/cobra.go`, `runCobraCommand` — 60 lines, and the last of them is why §8 exists.
6. `glazed/pkg/cmds/logging/section.go` — the model for the client section in §6, and already a dependency of this repository.